

DAY 4 | Kubernetes Backup & Disaster Recovery

Hands-on PostgreSQL 18 and pgBackRest • namespace postgres-ha

Topology seen in your screenshot: acid-postgres-ha-0/1/2, acid-postgres-ha-pooler (2 pods), and postgres-exporter. The Zalando operator owns acid-postgres-ha. This manual inspects that cluster, then creates an independent **backup-lab** pod with two PVCs for all pgBackRest and PITR exercises. Do not apply the restore commands to acid-postgres-ha PVCs.

Where: Ubuntu workstation or Windows PowerShell with kubectl; commands below use Bash on your Ubuntu workstation. **Time:** about 3-4 hours. **Storage:** a working default StorageClass, two PVCs (2Gi data, 3Gi repo), image pull access and root in lab containers. Confirm resource quotas and cluster policy.

1. Core concepts

Logical backup: pg_dump/pg_restore copy a selected database. **Physical backup:** all cluster files with WAL, allowing entire-cluster restore. **WAL archive:** PostgreSQL pushes completed WAL segments to a backup repository. **PITR:** base backup + WAL replay to a timestamp. Full copies a complete base; differential covers changes since the most recent full; incremental covers changes since the latest backup.

The Zalando Postgres Operator documents WAL-G/WAL-E archiving and restore integration. A pgBackRest exercise on that operator requires an intentional image, repository and Patroni configuration design. This training pod lets you execute every pgBackRest command now, without changing the operator-managed cluster.

2. Inspect existing Kubernetes deployment (read-only)

```
kubectl get pods -n postgres-ha -o wide
kubectl get svc -n postgres-ha
kubectl get postgresql -n postgres-ha
kubectl get pvc -n postgres-ha
kubectl exec -n postgres-ha acid-postgres-ha-0 -- \
  sh -c "command -v pgbackrest || true; command -v wal-g || true"
```

The primary service acid-postgres-ha and replica service acid-postgres-ha-repl appear in your screenshot. Service IPs can change; refer to service names. Inspect role on a pod with kubectl exec ... -- curl -s http://127.0.0.1:8008/; do not infer leadership from pod number.

3. Create a separate PostgreSQL/pgBackRest pod

3.1 Check storage and package availability

```
kubectl get storageclass
kubectl get ns postgres-ha
# PostgreSQL 18 image must support pgBackRest installed at boot.
# The lab startup checks pgbackrest version before starting PostgreSQL.
```

Save the entire manifest below as **backup-lab.yaml** on your Ubuntu workstation. It uses the default StorageClass. If none is default, set storageClassName on both claims. This is a disposable training setup: package installation occurs on pod start and needs apt/network access. For repeat use, build and pin an image containing PostgreSQL 18 and a compatible pgBackRest version.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: backup-lab-repo
  namespace: postgres-ha
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 3Gi
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: backup-lab-config
  namespace: postgres-ha
data:
  pgbackrest.conf: |
    [lab]
    pg1-path=/var/lib/postgresql/data/pgdata
    pg1-port=5432
```

```

[global]
repo1-path=/repo
repo1-retention-full=2
compress-type=gz
compress-level=3
log-level-console=info
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: backup-lab
  namespace: postgres-ha
spec:
  serviceName: backup-lab
  replicas: 1
  selector:
    matchLabels: {app: backup-lab}
  template:
    metadata:
      labels: {app: backup-lab}
    spec:
      initContainers:
        - name: permissions
          image: postgres:18-trixie
          command: [sh, -ec]
          args: ["chown -R 999:999 /repo /var/lib/postgresql/data"]
          securityContext: {runAsUser: 0}
          volumeMounts:
            - {name: repo, mountPath: /repo}
            - {name: data, mountPath: /var/lib/postgresql/data}
      containers:
        - name: postgres
          image: postgres:18-trixie
          command: [bash, -ec]
          args:
            - |
              apt-get update && apt-get install -y pgbackrest
              pgbackrest version
              exec docker-entrypoint.sh postgres \
                -c archive_mode=on \
                -c "archive_command=pgbackrest --stanza=lab archive-push %p"
          env:
            - {name: PGDATA, value: /var/lib/postgresql/data/pgdata}
            - {name: POSTGRES_PASSWORD, valueFrom: {secretKeyRef:
              {name: backup-lab-auth, key: password}}}
          volumeMounts:
            - {name: data, mountPath: /var/lib/postgresql/data}
            - {name: repo, mountPath: /repo}
            - {name: config, mountPath: /etc/pgbackrest}
          volumes:
            - {name: repo, persistentVolumeClaim: {claimName: backup-lab-repo}}
            - {name: config, configMap: {name: backup-lab-config}}
      volumeClaimTemplates:
        - metadata: {name: data}
          spec:
            accessModes: [ReadWriteOnce]
            resources:
              requests: {storage: 2Gi}

```

Create an independent password and the resources:

```

kubectl -n postgres-ha create secret generic backup-lab-auth \
  --from-literal=password="$(openssl rand -hex 20)"
kubectl apply -f backup-lab.yaml
kubectl -n postgres-ha rollout status statefulset/backup-lab \
  --timeout=10m
kubectl -n postgres-ha logs backup-lab-0 -c postgres --tail=25

```

If apt reports that pgBackRest does not support PostgreSQL 18, stop here and use an image that contains a version supporting PostgreSQL 18. If the pod is Pending, inspect kubectl describe pod and PVCs; common causes are missing StorageClass or capacity.

4. Configure stanza and archive WAL

```

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab stanza-create
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "SELECT pg_switch_wal();"

```

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c \
  "SELECT archived_count,failed_count FROM pg_stat_archiver;"
```

Expected: stanza-create and check succeed, archive_mode=on, archived_count increases. Repository and data PVCs must both be bound. The pod has no Service and is accessed only through kubectl exec.

5. Logical backup and restore

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "CREATE DATABASE drlab;"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "CREATE TABLE orders(id int PRIMARY KEY, note text);
  INSERT INTO orders VALUES (1,'full');"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pg_dump -U postgres -Fc -d drlab \
  -f /tmp/drlab.dump
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "CREATE DATABASE drlab_copy;"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pg_restore -U postgres -d drlab_copy \
  /tmp/drlab.dump
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab_copy -c "TABLE orders;"
```

Expected: drlab_copy includes row 1. The logical dump is in ephemeral /tmp: use kubectl cp if you need to keep it after pod replacement.

6. Full, incremental and differential backups

```
# Full: label ends F
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup
# Add row 2, then incremental: label ends I
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "INSERT INTO orders VALUES (2,'incremental');"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=incr backup
# Add row 3, then differential: label ends D
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "INSERT INTO orders VALUES (3,'differential');"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=diff backup
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
```

Confirm the F, I and D labels, their sizes, and dependency relationships in info.

7. Recovery workflow on the separate lab PVC

Why a recovery pod: pgBackRest restore requires PostgreSQL stopped. Scale backup-lab to zero, wait for the pod to disappear, then mount its data PVC and repository PVC in a separate utility pod. Restore to an empty data directory; move the old directory aside for the lab, allowing space for both copies. Scale up only after the utility pod has terminated.

7.1 Force archive and stop training pod

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "INSERT INTO orders VALUES (4,'archived WAL');"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "SELECT pg_switch_wal();"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check
kubectl -n postgres-ha scale statefulset/backup-lab \
  --replicas=0
kubectl -n postgres-ha wait --for=delete pod/backup-lab-0 \
  --timeout=180s
```

7.2 Create temporary recovery pod

Save as **backup-lab-recovery.yaml**. It uses the same image and package installation pattern as the training pod.

```
apiVersion: v1
```

```

kind: Pod
metadata:
  name: backup-lab-recovery
  namespace: postgres-ha
spec:
  restartPolicy: Never
  containers:
  - name: toolbox
    image: postgres:18-trixie
    command: [bash, -ec]
    args: ["apt-get update && apt-get install -y pgbackrest && sleep infinity"]
    volumeMounts:
    - {name: data, mountPath: /var/lib/postgresql/data}
    - {name: repo, mountPath: /repo}
    - {name: config, mountPath: /etc/pgbackrest}
  volumes:
  - {name: data, persistentVolumeClaim: {claimName: data-backup-lab-0}}
  - {name: repo, persistentVolumeClaim: {claimName: backup-lab-repo}}
  - {name: config, configMap: {name: backup-lab-config}}

kubectl apply -f backup-lab-recovery.yaml
kubectl -n postgres-ha wait --for=condition=Ready \
  pod/backup-lab-recovery --timeout=10m

```

7.3 Restore latest backup and archived WAL

```

kubectl -n postgres-ha exec backup-lab-recovery -- \
  bash -ec "mv /var/lib/postgresql/data/pgdata \
  /var/lib/postgresql/data/pgdata.before-restore && \
  install -d -o postgres -g postgres \
  /var/lib/postgresql/data/pgdata"
kubectl -n postgres-ha exec backup-lab-recovery -- \
  gosu postgres pgbackrest --stanza=lab \
  --type=default --target-action=promote restore
kubectl -n postgres-ha delete pod backup-lab-recovery \
  --wait=true
kubectl -n postgres-ha scale statefulset/backup-lab \
  --replicas=1
kubectl -n postgres-ha rollout status statefulset/backup-lab \
  --timeout=10m
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c "TABLE orders;"

```

Rows 1-4 should appear after archived WAL replay. If the recovery pod is still terminating, wait until it disappears before scaling up. Do not delete the old PGDATA copy until recovery is validated and the PVC capacity is known.

8. Point-in-time recovery exercise

8.1 Baseline, record timestamp, simulate DELETE

```

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
# Write the NEW full label ending F in your lab notes.
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "INSERT INTO orders VALUES (5, 'good row');"
TARGET_TIME=$(kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -Atqc \
  "SELECT to_char(clock_timestamp()) AT TIME ZONE 'UTC',
  'YYYY-MM-DD HH24:MI:SS.US') || '+00';")
printf "PITR target: %s\n" "$TARGET_TIME"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "DELETE FROM orders;"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "SELECT pg_switch_wal();"
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check

```

Keep the same Ubuntu terminal open: TARGET_TIME is a local shell variable. Record it separately if you change terminal. It must be after the row 5 insert and before DELETE.

8.2 Stop, restore to target, verify

Repeat section 7.1 scale to zero and 7.2 recovery pod creation. Remove only the previously restored PGDATA directory by moving it aside. Ensure PVC capacity; remove an older saved copy only after checking what it contains. Replace FULL_LABEL_HERE with

the full label recorded above.

```
kubectl -n postgres-ha scale statefulset/backup-lab \
  --replicas=0
kubectl -n postgres-ha wait --for=delete pod/backup-lab-0 \
  --timeout=180s
kubectl apply -f backup-lab-recovery.yaml
kubectl -n postgres-ha wait --for=condition=Ready \
  pod/backup-lab-recovery --timeout=10m
kubectl -n postgres-ha exec backup-lab-recovery -- \
  bash -ec "mv /var/lib/postgresql/data/pgdata \
  /var/lib/postgresql/data/pgdata.before-pitr && \
  install -d -o postgres -g postgres \
  /var/lib/postgresql/data/pgdata"
kubectl -n postgres-ha exec backup-lab-recovery -- \
  gosu postgres pgbackrest --stanza=lab \
  --set=FULL_LABEL_HERE --type=time \
  "--target=$TARGET_TIME" --target-action=promote restore
kubectl -n postgres-ha delete pod backup-lab-recovery \
  --wait=true
kubectl -n postgres-ha scale statefulset/backup-lab \
  --replicas=1
kubectl -n postgres-ha rollout status statefulset/backup-lab \
  --timeout=10m
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab \
  -c "SELECT * FROM orders ORDER BY id;"
```

Expected: rows 1-5 present; DELETE not replayed. Check `pg_is_in_recovery()` is false. Recovery creates a new PostgreSQL timeline. If restore cannot find WAL, check `archived_count`, `pgBackRest` info, log output and the target timestamp.

9. Compression, encryption, retention and cleanup

The lab config enables gzip compression and two-full-backup retention. Encryption must be configured before creating a fresh repository, using `repo1-cipher-type=aes-256-cbc` and a securely stored `repo1-cipher-pass`; do not change the existing lab repo midway. A production Kubernetes design should use remote object storage, credentials in a Secret, monitored WAL archiving and periodic restore tests.

Clean up **only** when the training exercise is finished. Deleting the PVCs permanently deletes lab data and backups.

```
kubectl -n postgres-ha scale statefulset/backup-lab --replicas=0
kubectl -n postgres-ha wait --for=delete pod/backup-lab-0 \
  --timeout=180s
kubectl -n postgres-ha delete statefulset backup-lab
kubectl -n postgres-ha delete configmap backup-lab-config
kubectl -n postgres-ha delete secret backup-lab-auth
# Optional only after confirming you no longer need the backups:
# kubectl -n postgres-ha delete pvc \
#   data-backup-lab-0 backup-lab-repo
```

References

`pgBackRest` user guide: <https://pgbackrest.org/user-guide.html>

Zalando operator administrator guide: <https://postgres-operator.readthedocs.io/en/latest/administrator/>

Kubernetes StatefulSets: <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>

PostgreSQL 18 continuous archiving: <https://www.postgresql.org/docs/18/continuous-archiving.html>